BRACT's

Vishwakarma Institute of Technology, Pune

Practical Implementation Sheet

Department: CSE-AI	Semester: 5	Academic Year: 2026-27
Division: C	Batch: 3	Practical No: 1
Roll no: 68	PRN No: 12415154	Name of Student: Kalpesh Patil

Subject Name: Deep Learning

Problem Statement:

Design and implement a Multilayer Perceptron (MLP) for classification of the Iris or Wine dataset, and evaluate its performance using accuracy and a confusion matrix.

Algorithm:

Multilayer Perceptron (MLP) for Iris/Wine Dataset Classification

1. **Import** the required libraries.
2. **Load** the Iris (or Wine) dataset.
3. **Split** the dataset into training and testing sets.
4. **Create** an MLP classifier by specifying:
    - Hidden layers
    - Activation function (ReLU)
    - Optimizer (Adam)
    - Number of iterations
5. **Train** the MLP model using the training data.
6. **Predict** the class labels for the test data.
7. **Calculate** the accuracy of the model using the predicted and actual labels.

8. **Generate** the confusion matrix to compare actual and predicted classes.
9. **Display** the accuracy, confusion matrix, and classification report.
10. **Analyse** the performance of the MLP classifier.

Code:

```python
import numpy as np
from sklearn import datasets
from sklearn.model_selection import train_test_split
from sklearn.metrics import accuracy_score, confusion_matrix, classification_report
from sklearn.preprocessing import StandardScaler
from sklearn.neural_network import MLPClassifier
from sklearn.model_selection import cross_val_score
from sklearn.metrics import f1_score, accuracy_score, precision_score, recall_score
import matplotlib.pyplot as plt
from sklearn.metrics import ConfusionMatrixDisplay
```

Python

```python
# Load the Wine dataset
wine = datasets.load_wine()
X = wine.data
y = wine.target

# First scenario: 70% training, 30% test
X_train1, X_test1, y_train1, y_test1 = train_test_split(X, y, test_size=0.3, random_state=42)

# Second scenario: 90% training, 10% test
X_train2, X_test2, y_train2, y_test2 = train_test_split(X, y, test_size=0.1, random_state=42)

# MLP1 100-100
mlp1 = MLPClassifier(hidden_layer_sizes=(100, 100), max_iter=2000, learning_rate_init=0.001, random_state=42)

# MLP2 50-
mlp2 = MLPClassifier(hidden_layer_sizes=(50,), max_iter=2000, learning_rate_init=0.001, random_state=42)
```

Python

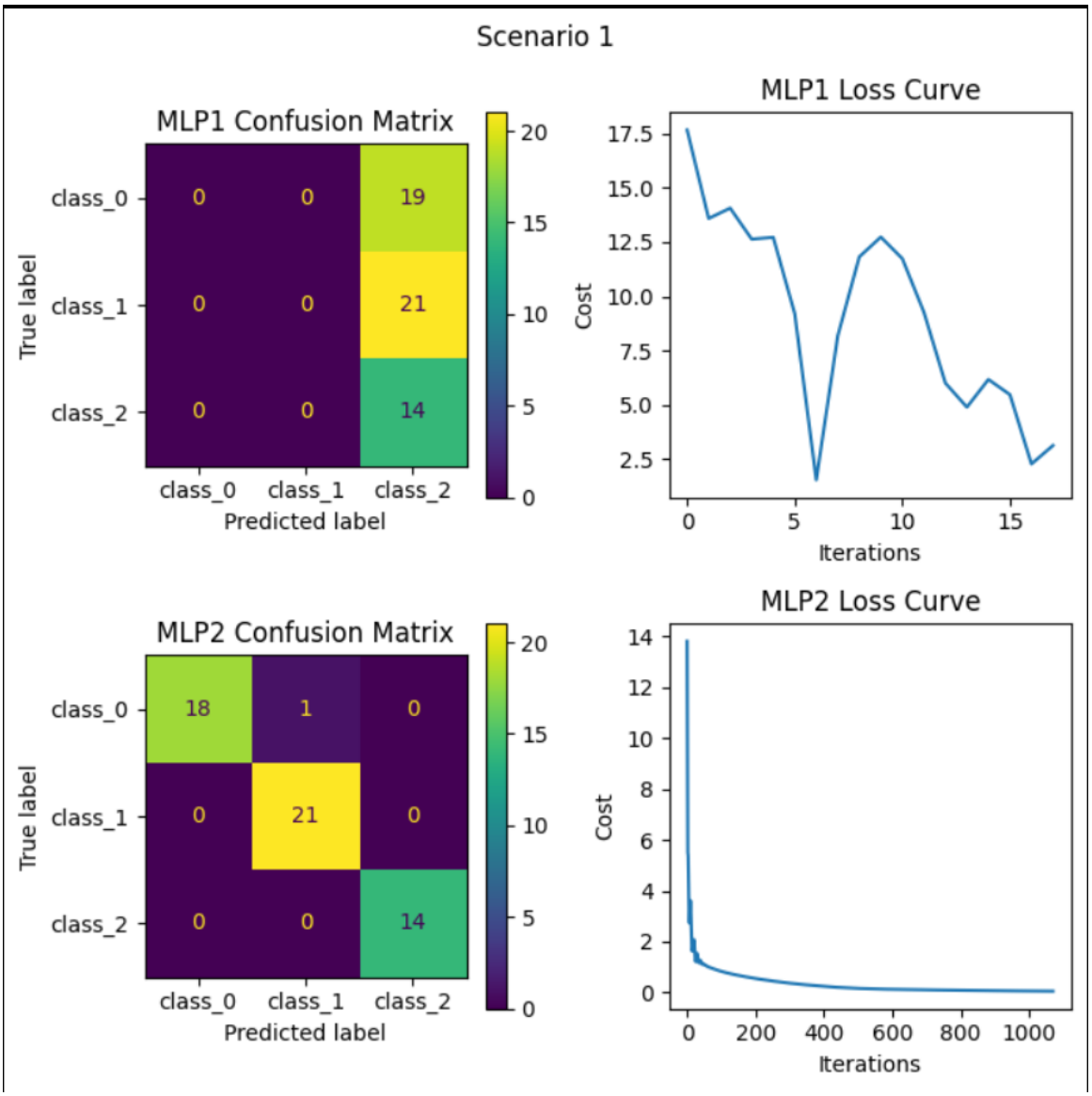
```python
def calculate_and_plot_metrics(X_train, y_train, X_test, y_test, title):
    mlp1.fit(X_train, y_train)
    y_pred_mlp1 = mlp1.predict(X_test)

    mlp2.fit(X_train, y_train)
    y_pred_mlp2 = mlp2.predict(X_test)

    # Calculate metrics
    def calculate_metrics(y_true, y_pred):
        accuracy = accuracy_score(y_true, y_pred)
        f1 = f1_score(y_true, y_pred, average='weighted')
        precision = precision_score(y_true, y_pred, average='weighted', zero_division=1)
        recall = recall_score(y_true, y_pred, average='weighted')
        return accuracy, f1, precision, recall

    # Metrics for MLP1
    accuracy_mlp1, f1_mlp1, precision_mlp1, recall_mlp1 = calculate_metrics(y_test, y_pred_mlp1)

    # Metrics for MLP2
    accuracy_mlp2, f1_mlp2, precision_mlp2, recall_mlp2 = calculate_metrics(y_test, y_pred_mlp2)

    # Confusion Matrix and Visualization (MLP1)
    cm_mlp1 = confusion_matrix(y_test, y_pred_mlp1)
    disp_mlp1 = ConfusionMatrixDisplay(confusion_matrix=cm_mlp1, display_labels=wine.target_names)
    fig, axes = plt.subplots(2, 2, figsize=(7, 7))
    disp_mlp1.plot(cmap='viridis', values_format='d', ax=axes[0, 0])
    axes[0, 0].set_title("MLP1 Confusion Matrix")

    # Loss Curve (MLP1)
    axes[0, 1].plot(mlp1.loss_curve_)
    axes[0, 1].set_title("MLP1 Loss Curve")
    axes[0, 1].set_xlabel('Iterations')
    axes[0, 1].set_ylabel('Cost')
```

```python
# Confusion Matrix and Visualization (MLP2)
cm_mlp2 = confusion_matrix(y_test, y_pred_mlp2)
disp_mlp2 = ConfusionMatrixDisplay(confusion_matrix=cm_mlp2, display_labels=wine.target_names)
disp_mlp2.plot(cmap='viridis', values_format='d', ax=axes[1, 0])
axes[1, 0].set_title("MLP2 Confusion Matrix")

# Loss Curve (MLP2)
axes[1, 1].plot(mlp2.loss_curve_)
axes[1, 1].set_title("MLP2 Loss Curve")
axes[1, 1].set_xlabel('Iterations')
axes[1, 1].set_ylabel('Cost')

# Calculate 10-fold cross-validation results
cross_val_scores1_mlp1 = cross_val_score(mlp1, X_train, y_train, cv=10, scoring='f1_weighted')
cross_val_scores1_mlp2 = cross_val_score(mlp2, X_train, y_train, cv=10, scoring='f1_weighted')

plt.suptitle(title)
plt.tight_layout()
plt.show()

return accuracy_mlp1, f1_mlp1, precision_mlp1, recall_mlp1, accuracy_mlp2, f1_mlp2, precision_mlp2, recall_mlp2, cross_val_scores1_mlp1,
```

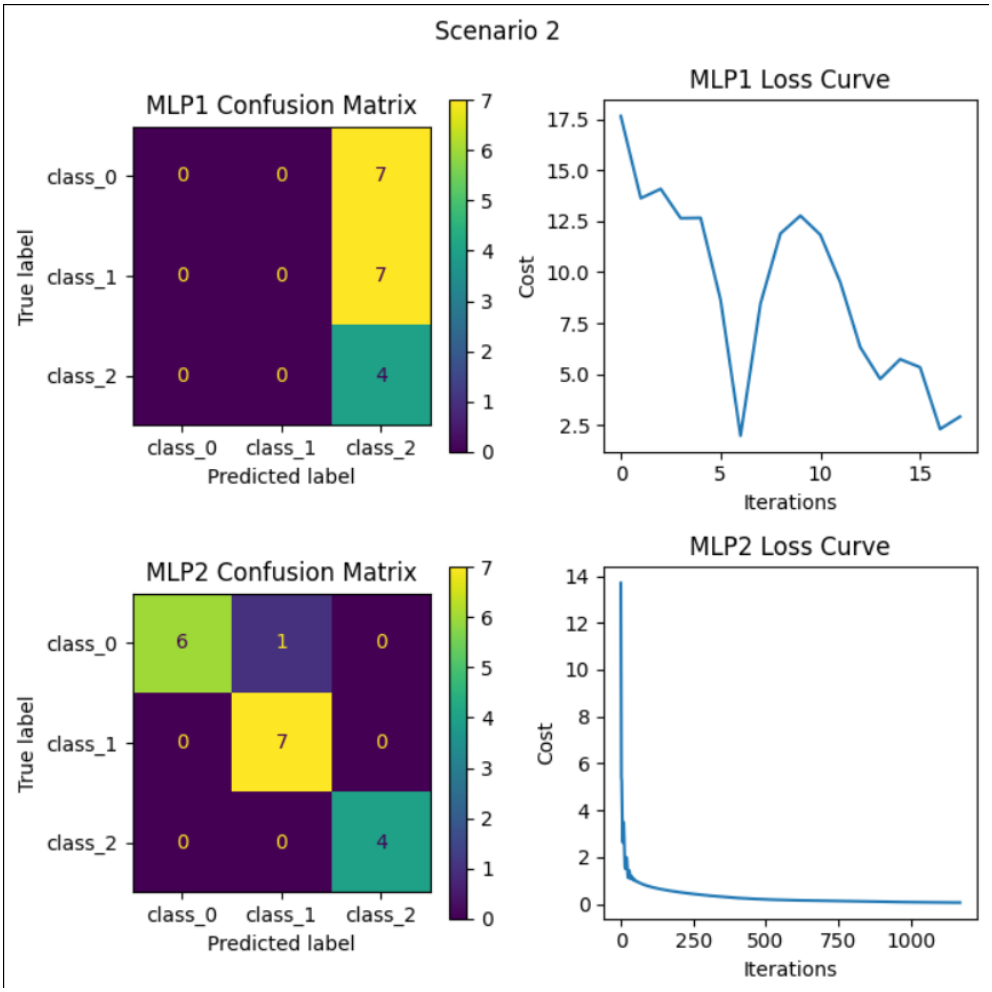
```python
# Calculations for Scenario 1
accuracy1_mlp1, f1_1_mlp1, precision1_mlp1, recall1_mlp1, accuracy1_mlp2, f1_1_mlp2, precision1_mlp2, recall1_mlp2, cross_val_scores1_mlp1, cross_val_scores1_mlp2 = calculate_and_p

# Outputs for Scenario 1
print("Scenario 1 (MLP1):")
print(f"Accuracy: {accuracy1_mlp1:.4f}\nF1 Score: {f1_1_mlp1:.4f}\nPrecision: {precision1_mlp1:.4f}\nRecall: {recall1_mlp1:.4f}\n")
print("Scenario 1 (MLP2):")
print(f"Accuracy: {accuracy1_mlp2:.4f}\nF1 Score: {f1_1_mlp2:.4f}\nPrecision: {precision1_mlp2:.4f}\nRecall: {recall1_mlp2:.4f}\n")
print("Scenario 1 10-Fold Cross-Validation (MLP1) F1 Score:", np.mean(cross_val_scores1_mlp1))
print("Scenario 1 10-Fold Cross-Validation (MLP2) F1 Score:", np.mean(cross_val_scores1_mlp2))

# Calculations for Scenario 2
accuracy2_mlp1, f1_2_mlp1, precision2_mlp1, recall2_mlp1, accuracy2_mlp2, f1_2_mlp2, precision2_mlp2, recall2_mlp2, cross_val_scores2_mlp1, cross_val_scores2_mlp2 = calculate_and_p

# Outputs for Scenario 2
print("Scenario 2 (MLP1):")
print(f"Accuracy: {accuracy2_mlp1:.4f}\nF1 Score: {f1_2_mlp1:.4f}\nPrecision: {precision2_mlp1:.4f}\nRecall: {recall2_mlp1:.4f}\n")
print("Scenario 2 (MLP2):")
print(f"Accuracy: {accuracy2_mlp2:.4f}\nF1 Score: {f1_2_mlp2:.4f}\nPrecision: {precision2_mlp2:.4f}\nRecall: {recall2_mlp2:.4f}\n")
print("Scenario 2 10-Fold Cross-Validation (MLP1) F1 Score:", np.mean(cross_val_scores2_mlp1))
print("Scenario 2 10-Fold Cross-Validation (MLP2) F1 Score:", np.mean(cross_val_scores2_mlp2))

# Scenario 1 and Scenario 2 comparison
print("Comparison:")
print("Scenario 1 (MLP1) Accuracy - Scenario 2 (MLP1) Accuracy:", accuracy1_mlp1 - accuracy2_mlp1)
print("Scenario 1 (MLP2) Accuracy - Scenario 2 (MLP2) Accuracy:", accuracy1_mlp2 - accuracy2_mlp2)
print("Scenario 1 (MLP1) F1 Score - Scenario 2 (MLP1) F1 Score:", f1_1_mlp1 - f1_2_mlp1)
print("Scenario 1 (MLP2) F1 Score - Scenario 2 (MLP2) F1 Score:", f1_1_mlp2 - f1_2_mlp2)
```

(variable) accuracy2_mlp2: Float

Output:




```
Scenario 1 (MLP1):
Accuracy: 0.2593
F1 Score: 0.1068
Precision: 0.8080
Recall: 0.2593

Scenario 1 (MLP2):
Accuracy: 0.9815
F1 Score: 0.9814
Precision: 0.9823
Recall: 0.9815

Scenario 1 10-Fold Cross-Validation (MLP1) F1 Score: 0.2315341521223874
Scenario 1 10-Fold Cross-Validation (MLP2) F1 Score: 0.9346122396122396
```




```
Scenario 2 (MLP1):
Accuracy: 0.2222
F1 Score: 0.0808
Precision: 0.8272
Recall: 0.2222

Scenario 2 (MLP2):
Accuracy: 0.9444
F1 Score: 0.9442
Precision: 0.9514
Recall: 0.9444

Scenario 2 10-Fold Cross-Validation (MLP1) F1 Score: 0.22674655388471177
Scenario 2 10-Fold Cross-Validation (MLP2) F1 Score: 0.9561985236985237
Comparison:
Scenario 1 (MLP1) Accuracy - Scenario 2 (MLP1) Accuracy: 0.037037037037037035
Scenario 1 (MLP2) Accuracy - Scenario 2 (MLP2) Accuracy: 0.03703703703703709
Scenario 1 (MLP1) F1 Score - Scenario 2 (MLP1) F1 Score: 0.02594573182808478
Scenario 1 (MLP2) F1 Score - Scenario 2 (MLP2) F1 Score: 0.0372870186823675
```